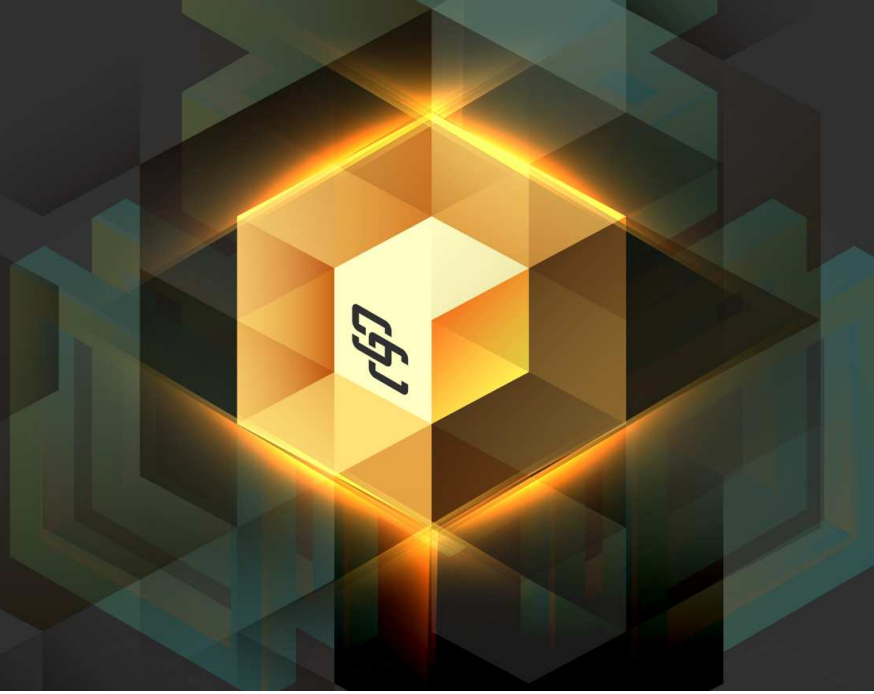




Security Audit

Full Audit Report For MetaMemeVault



Client: MetaMemeVault

<https://www.metamemevault.com>

October 7, 2025

Version: 1.1

Table of Contents

- 1 Disclaimer 3
- 2 Security Assessment Procedure 4
- 3 Risk Rating 5
- 4 Category 6
- 5 Executive Summary 7
- 6 Audit Information 8
- 7 Findings List 12
- 8 Findings 13
- A Appendix 27
 - A.1 Source Units in Scope 27
 - A.2 Visibility, Mutability, Modifier function testing 28
 - A.3 Contracts Description Table 29
 - A.4 Call Graph 36
 - A.5 UML Class Diagram 37



1 Disclaimer

This security assessment does not guarantee the security of the program instruction received from the client, herein referred to as "Source code."

The Service Provider will not be held liable for any legal liability arising from errors in the security assessment. The responsibility for ensuring the security of the program instruction will be solely with the Client, herein referred to as "Service User." The Service User agrees not to hold the Service Provider liable for any such liability.

By contract, the Service Provider is committed to conducting security assessments with integrity, professional ethics, and transparency in delivering security assessments to users. The Service Provider reserves the right to postpone the delivery of the security assessment if necessary, regardless of the reason for the delay. The Service Provider will not be held responsible for any delayed security assessments.

If the Service Provider identifies a vulnerability, we will notify the Service User via a Preliminary Report, which will be maintained in confidence for security purposes. The Service Provider disclaims responsibility in the event of any attacks occurring before or after conducting a security assessment. All responsibility for ensuring the security of the program instruction will be solely with the Service User.

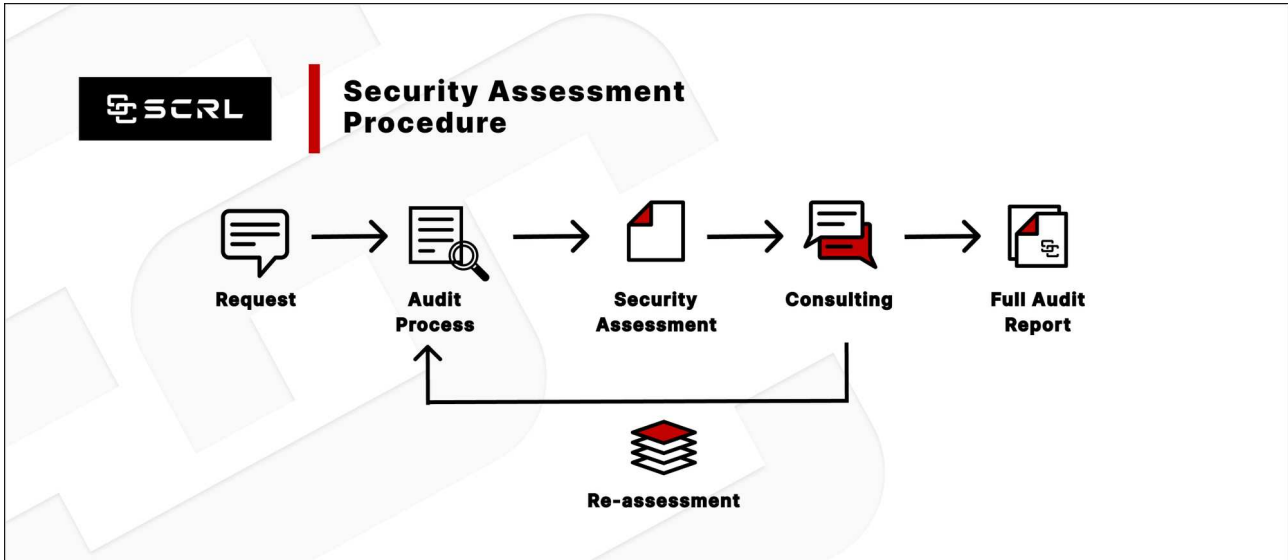
Please be advised that the Security Assessment is not an investment advisory document and should not be interpreted as such. We hereby disclaim any liability arising from any investment:

- Security Assessment Is **Not Financial/Investment Advice** Any loss arising from any investment in any project is the responsibility of the investor.
- SCRL **disclaims** any liability incurred, whether it's **Rugpull, Abandonment, Soft Rugpull, Exploit, Exit Scam.**
- **Cryptocurrency and digital token involve high risks; investors may lose all investment money and should study information carefully and make investments according to own risk profile.**
- Please thoroughly comprehend any aspect of a project, including liquidity checks and pre-sale stages, and assume the associated risks independently. We hereby disclaim any liability for any investment decisions made by you.

For full Legal / TOS / Privacy Policy please visit: <https://scrl.io/legal>



2 Security Assessment Procedure



- **Request:** The client must submit a formal request and follow the procedure. By submitting the source code and agreeing to the terms of service.
- **Audit Process:** Check for vulnerabilities and vulnerabilities from source code obtained by experts using formal verification methods, including using powerful tools such as Static Analysis, SWC Registry, Dynamic Security Analysis, Automated Security Tools, CWE, Syntax & Parameter Check with AI ,WAS (Warning Avoidance System a python script tools powered by SCRL).
- **Security Assessment:** Deliver Preliminary Security Assessment to clients to acknowledge the risks and vulnerabilities.
- **Consulting:** Discuss on risks and vulnerabilities encountered by clients to apply to their source code to mitigate risks.
- **Re-assessment:** Reassess the security when the client implements the source code improvements and if the client is satisfied with the results of the audit. We will proceed to the next step.
- **Full Audit Report:** SCRL provides clients with official security assessment reports informing them of risks and vulnerabilities. Officially and it is assumed that the client has been informed of all the information.



3 Risk Rating

Risk rating using this commonly defined: **Risk rating = impact * confidence**

- **Impact:** The severity and potential impact of an attacker attack
- **Confidence:** Ensuring that attackers expose and use this vulnerability

Confidence & Impact [Likelihood]	Low	Medium	High
Low	Informational	Low	Medium
Medium	Low	Medium	High
High	Medium	High	Critical

Severity is a risk assessment It is calculated from the Impact and Confidence values using the following calculation methods, Risk rating = impact * confidence It is categorized into 7 categories severity based

Severity Risk	Description
Critical	Critical severity is assigned to security vulnerabilities that pose a severe threat to the smart contract and the entire blockchain ecosystem.
High	High-severity issues should be addressed quickly to reduce the risk of exploitation and protect users' funds and data.
Medium	It's essential to fix medium-severity issues in a reasonable timeframe to enhance the overall security of the smart contract.
Low	While low-severity issues can be less urgent, it's still advisable to address them to improve the overall security posture of the smart contract.
Informational	Used to categorize security findings that do not pose a direct security threat to the smart contract or its users. Instead, these findings provide additional information, recommendations



4 Category

Category	Description
Centralization	Centralization Risk is The risk incurred by a sole proprietor, such as the Owner being able to change something without permission
Economics Risk	Economics Risk is Risks that may affect the economic mechanism system, such as the ability to increase Mint token
Logical Issue	Logical Issue is that can cause errors to core processing, such as any prior operations that cause background processes to crash.
Authorization	Authorization is Possible pitfalls from weak coding allows unrelated people to take any action to modify the values.
Mathematical	Any erroneous arithmetic operations affect the operation of the system or lead to erroneous values.
Naming Conventions	naming variables that may affect code understanding or naming inconsistencies
Security Risk	Security Risk of loss or damage if it's no mitigate
Coding Style	Coding Style is Tips coding for efficiency performance
Best Practices	Best Practices is suggestions for improvement
Optimization	Optimization is performance improvement
Gas Optimization	Gas Optimization is increase performance to avoid expensive gas
Dead Code	Dead Code having unused code This may result in wasted resources and gas fees.
Input Validation	Proper input validation is essential to ensure that a smart contract processes only valid and anticipated data. Failure to implement robust input validation mechanisms may lead to various security vulnerabilities, including logic manipulation, unauthorized access, and unintended contract behavior.



5 Executive Summary

For this security assessment, SCRL received a request on **October 1, 2025**



Audit Score	Project Website	X (Twitter)	Telegram
8.7	https://www.metamemevault.com	https://x.com/metamemevault	https://t.me/metamemevault_chat

Client	Confidential	Audit Method	Language
MetaMemeVault	Public	Static Analysis + Dynamic Analysis + Manual Review + Formal Verification + Hyper Fuzzing + WAS (Custom Detector)	Solidity

Vulnerabilities Summary:



10
Total Findings

0
Resolved

0
Mitigated

10
Acknowledge

0
Declined

0

Critical

Critical severity is assigned to security vulnerabilities that pose a severe threat to the smart contract and the entire blockchain ecosystem.

0

High

High-severity issues should be addressed quickly to reduce the risk of exploitation and protect users funds and data.

2

Medium

Acknowledge: 2

It essential to fix medium-severity issues in a reasonable timeframe to enhance the overall security of the smart contract.

1

Low

Acknowledge: 1

While low-severity issues can be less urgent, it still advisable to address them to improve the overall security posture of the smart contract.

7

Informational

Acknowledge: 7

Used to categorize security findings that do not pose a direct security threat to the smart contract or its users. Instead, these findings provide information



6 Audit Information

Audit Scope:

File	SHA1-Hash
src/ PresaleContracts.sol	0e0d169476d4729de8b0ca66e644ffe5d21bd234
src/MetaMemeVault.sol	066bcbff5d1c5395f09ba6a29aff47eba3289ba0

Onchain Scope:

Chain	Contract Address
Ethereum	0xc53fa85b734717cfd999343f6024165f0ec423b7
Ethereum	0x8e737f5e206d5e94c2890e83b2d4df82fc7c089c

Security Assessment Author:

Name	Role
Chinnakit J.	CEO & Founder
Ronny C.	CTO & Head of Security Researcher
Mark K.	Security Researcher

Smart Contract Audit Summary:



The results of the security assessment revealed

No Critical Vulnerabilities.

Full Audit Report For MetaMemeVault
October 7, 2025



Digital Sign:



ID: 7BDC60C8-C8DF-4DD8-9B6C-67D549A44C5F
Digitally signed by <contact@scrl.io>
October 07, 2025 10:06 AM +07



SWC Checklist

ID	Title	Scanning	Result
SWC-100	Function Default Visibility	Completed	No Risk
SWC-101	Integer Overflow and Underflow	Completed	No Risk
SWC-102	Outdated Compiler Version	Completed	No Risk
SWC-103	FloatingPragma	Completed	No Risk
SWC-104	Unchecked Call Return Value	Completed	No Risk
SWC-105	Unprotected Ether Withdrawal	Completed	No Risk
SWC-106	Unprotected SELFDESTRUCT Instruction	Completed	No Risk
SWC-107	Reentrancy	Completed	No Risk
SWC-108	State Variable Default Visibility	Completed	No Risk
SWC-109	Uninitialized Storage Pointer	Completed	No Risk
SWC-110	Assert Violation	Completed	No Risk
SWC-111	Use of Deprecated Solidity Functions	Completed	No Risk
SWC-112	Delegatecall to Untrusted Callee	Completed	No Risk
SWC-113	DoS with Failed Call	Completed	No Risk
SWC-114	Transaction Order Dependence	Completed	No Risk
SWC-115	Authorization through tx.origin	Completed	No Risk
SWC-116	Block values as a proxy for time	Completed	No Risk
SWC-117	Signature Malleability	Completed	No Risk
SWC-118	Incorrect Constructor Name	Completed	No Risk
SWC-119	Shadowing State Variables	Completed	No Risk
SWC-120	Weak Sources of Randomness from Chain Attributes	Completed	No Risk
SWC-121	Missing Protection against Signature Replay Attacks	Completed	No Risk



ID	Title	Scanning	Result
SWC-122	Lack of Proper Signature Verification	Completed	No Risk
SWC-123	Requirement Violation	Completed	No Risk
SWC-124	Write to Arbitrary Storage Location	Completed	No Risk
SWC-125	Incorrect Inheritance Order	Completed	No Risk
SWC-126	Insufficient Gas Griefing	Completed	No Risk
SWC-127	Arbitrary Jump with Function Type Variable	Completed	No Risk
SWC-128	DoS With Block Gas Limit	Completed	No Risk
SWC-129	Typographical Error	Completed	No Risk
SWC-130	Right-To-Left-Override control character (U+202E)	Completed	No Risk
SWC-131	Presence of unused variables	Completed	No Risk
SWC-132	Unexpected Ether balance	Completed	No Risk
SWC-133	Hash Collisions With Multiple Variable Length Arguments	Completed	No Risk
SWC-134	Message call with hardcoded gas amount	Completed	No Risk
SWC-135	Code With No Effects	Completed	No Risk
SWC-136	Unencrypted Private Data On-Chain	Completed	No Risk



OWASP Smart Contract Top 10 2025 Checklist

ID	Title	Scanning	Result
SC01:2025	Access Control Vulnerabilities	Completed	No Risk
SC02:2025	Price Oracle Manipulation	Completed	No Risk
SC03:2025	Logic Errors	Completed	No Risk
SC04:2025	Lack of Input Validation	Completed	No Risk
SC05:2025	Reentrancy Attacks	Completed	No Risk
SC06:2025	Unchecked External Calls	Completed	No Risk
SC07:2025	Flash Loan Attacks	Completed	No Risk
SC08:2025	Integer Overflow and Underflow	Completed	No Risk
SC09:2025	Insecure Randomness	Completed	No Risk
SC10:2025	Denial of Service (DoS) Attacks	Completed	No Risk



7 Findings List



0
Critical

0
High

2
Medium

1
Low

7
Informational

This security assessment report for **MetaMemeVault** a total of **10 vulnerabilities**. The evaluation was conducted using the **Static Analysis + Dynamic Analysis + Manual Review + Formal Verification + Hyper Fuzzing + WAS (Custom Detector)** security assessment methodology.

ID	Vulnerabilities	Severity	Category	Status
DBM-01	Imprecise arithmetic operations order (divide-before-multiply)	Medium	Mathematical	Acknowledge
CEN-01	Centralization Risk	Medium	Centralization	Acknowledge
MEA-01	Missing Events for Arithmetic State Changes (events-maths)	Low	Best Practices	Acknowledge
SNC-01	Conformity to Solidity naming conventions	Informational	Coding Style	Acknowledge
CNN-01	Conformance to Numeric Notation Best Practices (too-many-digits)	Informational	Coding Style	Acknowledge
UCE-01	Use Custom Errors	Informational	Gas Optimization	Acknowledge
DIV-01	Don't initialize variables with default value	Informational	Gas Optimization	Acknowledge
PDM-01	Use of Post-Increment (i++) Instead of Pre-Increment (++) in Loops	Informational	Gas Optimization	Acknowledge
IRU-01	Inefficient require() Usage with && Conditions	Informational	Gas Optimization	Acknowledge
UIC-01	Use != 0 instead of > 0 for unsigned integer comparison	Informational	Gas Optimization	Acknowledge



8 Findings

DBM-01: Imprecise arithmetic operations order (divide-before-multiply)

Vulnerabilities	Severity	Category	Status
Imprecise arithmetic operations order (divide-before-multiply)	Medium	Mathematical	Acknowledge

Finding Code

✗ `PresaleContracts.buyWithETH(address,uint256)` (`src/PresaleContracts.sol:153-180`) performs a multiplication on the result of a division:

- `baseTokens = (usdAmount * DECIMALS) / stage.pricePerTokenUSD` (`src/PresaleContracts.sol#168-170`)
- `bonusTokens = (baseTokens * stage.bonusPercent) / 100` (`src/PresaleContracts.sol#170-172`)

✗ `PresaleContracts.buyWithETH(address,uint256)` (`src/PresaleContracts.sol:153-180`) performs a multiplication on the result of a division:

- `usdAmount = (msg.value * ethPrice) / 1e8` (`src/PresaleContracts.sol#164-165`)
- `baseTokens = (usdAmount * DECIMALS) / stage.pricePerTokenUSD` (`src/PresaleContracts.sol#168-170`)

✗ `PresaleContracts.buyWithUSDT(uint256,address)` (`src/PresaleContracts.sol:181-204`) performs a multiplication on the result of a division:

- `baseTokens = (usdAmount * DECIMALS) / stage.pricePerTokenUSD` (`src/PresaleContracts.sol#198-200`)
- `bonusTokens = (baseTokens * stage.bonusPercent) / 100` (`src/PresaleContracts.sol#200-201`)

✗ `PresaleContracts.calculateTokens(uint256)` (`src/PresaleContracts.sol:401-413`) performs a multiplication on the result of a division:

- `baseTokens = (_usdAmount * DECIMALS) / s.pricePerTokenUSD` (`src/PresaleContracts.sol#409`)
- `bonusTokens = (baseTokens * s.bonusPercent) / 100` (`src/PresaleContracts.sol#409-412`)

Description

The contract performs arithmetic operations in an order that can lead to loss of precision, particularly when performing division before multiplication on integer values. Solidity uses integer arithmetic, which means it does not support decimals and always rounds down toward zero when dividing.

When division is performed before multiplication, especially with small numerators or large divisors, valuable precision can be lost, potentially leading to incorrect calculations or unexpected outcomes in logic such as reward distributions, fee calculations, or percentage-based logic.

Recommendation

- Reorder arithmetic expressions to multiply before dividing when possible, to preserve accuracy in integer calculations.
- Consider using SafeMath (**for older Solidity versions**) or **built-in overflow checks (in Solidity $\geq 0.8.0$)** when rearranging operations.
- Add unit tests to verify expected behavior, especially for boundary conditions.

References



Solidity Docs – Types and Units: <https://docs.soliditylang.org/en/latest/types.html#mathematical-and-bitwise-operators>



CEN-01: Centralization Risk

Vulnerabilities	Severity	Category	Status
Centralization Risk	Medium	Centralization	Acknowledge

Finding Code

File: MetaMemeVault.sol

109: function recoverEth(address payable to, uint256 amount) external onlyOwner {

127: function recoverToken(address tokenAddress, address to, uint256 amount) external onlyOwner {

File: PresaleContracts.sol

152: function setToken(address _mmvToken) external onlyOwner {

254: function setClaimingConfig(bool _enabled,uint256 _claimStart,uint256 _bonusUnlock) external
onlyOwner {

264: function setPaymentWallet(address _wallet) external onlyOwner {

272: function setReferralGasLimit(uint256 _gasLimit) external onlyOwner {

281: function lockStage(uint8 _stage) external onlyOwner {

289: function advanceStage() external onlyOwner {

298: function overrideETHPrice(uint256 _price) external onlyOwner {

317: function updateStage(uint8 _stageIndex,uint256 _tokensToSell,uint256 _pricePerTokenUSD,uint256
_bonusPercent) external onlyOwner {

330: function withdrawUnsoldTokens(address _to) external onlyOwner {

360: function recoverTokens(address _token, address _to, uint256 _amount) external onlyOwner {

379: function setPaused(bool _paused) external onlyOwner {

Description

Centralization Risk



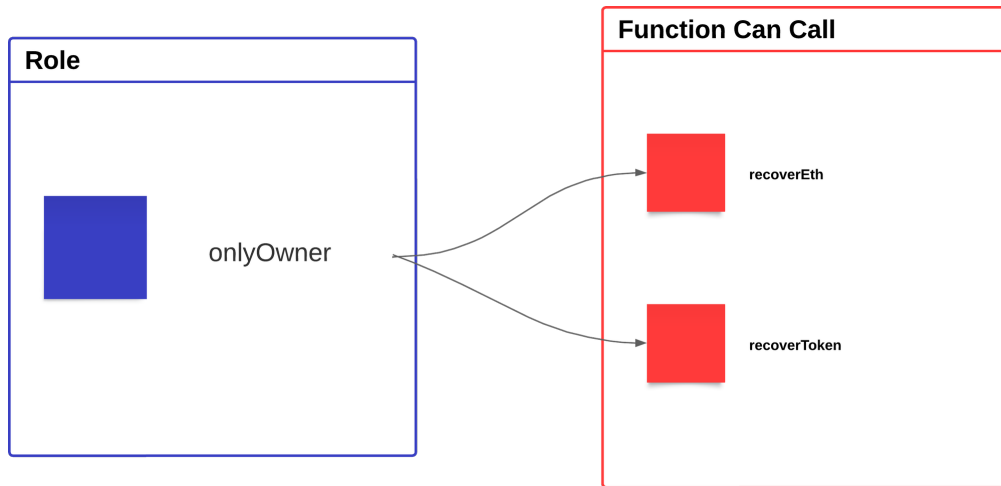


Figure 1 - This image shows who has privileges to call the function of contract MetaMemeVault.sol

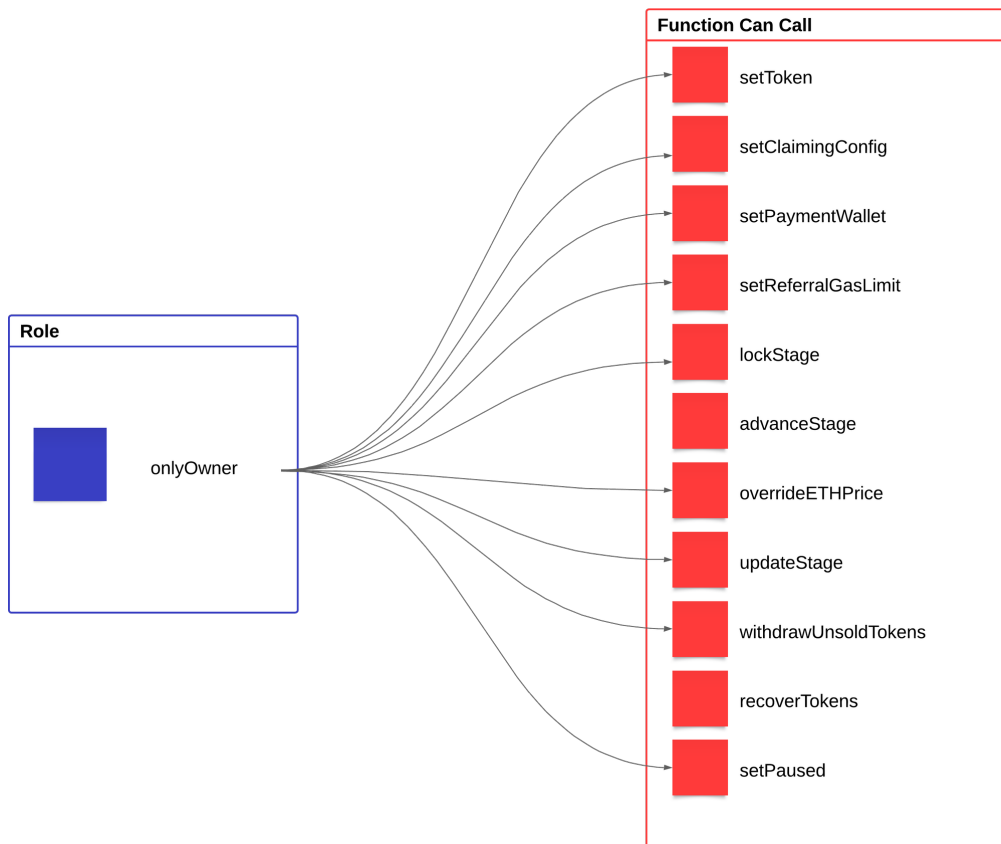


Figure 2 - This image shows who has privileges to call the function of contract PresaleContracts.sol

A Centralization Risk has been identified for the `Contract` entity, which has privileges set to `onlyOwner`. This restriction allows the execution of the following functions: `function`

Each function possesses the following capabilities:

MetaMemeVault.sol

- `recoverEth` Capability: Allows the owner to recover any ERC20 tokens from the contract and send them to a specified address.
- `recoverToken` Capability: Allows the owner to recover any ERC20 tokens from the contract and send them to a specified address.



PresaleContracts.sol

- `setToken` Capability: Allows the owner to set the MMV token address for the presale.
- `setClaimingConfig` Capability: Allows the owner to configure the claiming process, including enabling/disabling claiming and setting the start times for base and bonus token claims.
- `setPaymentWallet` Capability: Allows the owner to set the payment wallet where funds from the presale are sent.
- `setReferralGasLimit` Capability: Allows the owner to set the gas limit for referral registrations to ensure transactions are successful and cost-effective.
- `lockStage` Capability: Allows the owner to lock a specific presale stage, preventing further sales in that stage.
- `advanceStage` Capability: Allows the owner to manually advance the presale to the next stage.
- `overrideETHPrice` Capability: Allows the owner to manually override the ETH price feed, which is used for calculating the number of tokens sold based on the amount of ETH or USDT received.
- `updateStage` Capability: Allows the owner to update the details of a specific presale stage, including the number of tokens to sell, the price per token in USD, and the bonus percentage.
- `withdrawUnsoldTokens` Capability: Allows the owner to withdraw any unsold tokens from the contract after the presale has ended.
- `recoverTokens` Capability: Allows the owner to recover any ERC20 tokens from the contract and send them to a specified address.
- `setPaused` Capability: Allows the owner to pause or unpause the contract, which can be used to halt or resume the presale process and other critical functions.

Recommendation

In terms of timeframes, there are three categories: short-term, long-term, and permanent.

For short-term solutions, a combination of timelock and multi-signature (2/3 or 3/5) can be used to mitigate risk by delaying sensitive operations and avoiding a single point of failure in key management. This includes implementing a timelock with a reasonable latency, such as 48 hours, for privileged operations; assigning privileged roles to multi-signature wallets to prevent private key compromise; and sharing the timelock contract and multi-signer addresses with the public via a medium/blog link.

For long-term solutions, a combination of timelock and DAO can be used to apply decentralization and transparency to the system. This includes implementing a timelock with a reasonable latency, such as 48 hours, for privileged operations; introducing a DAO/governance/voting module to increase transparency and user involvement; and sharing the timelock contract, multi-signer addresses, and DAO information with the public via a medium/blog link.

Finally, permanent solutions should be implemented to ensure the ongoing security and protection of the system.

- Renounce the ownership and cannot turn back to claim a privileges to execution a function

OR

- Remove a contain centralization risk function and deployed a new contract

References

.



MEA-01: Missing Events for Arithmetic State Changes (events-maths)

Vulnerabilities	Severity	Category	Status
Missing Events for Arithmetic State Changes (events-maths)	Low	Best Practices	Acknowledge

Finding Code

✖

PresaleContracts.setClaimingConfig(bool,uint256,uint256) (src/PresaleContracts.sol:241-252) should emit an event for:

• claimStartTime = _claimStart (src/PresaleContracts.sol#249-250)

• bonusUnlockTime = _bonusUnlock (src/PresaleContracts.sol#251-252)

Description

The contract performs **arithmetic operations that modify critical state variables** without emitting corresponding **events**.

In Solidity, emitting events when state-changing arithmetic occurs is considered a **best practice** because it improves **on-chain transparency**, allows for easier **off-chain monitoring**, and facilitates debugging or analytics.

Without events, users and external systems (e.g., block explorers, dApps, monitoring tools) may find it difficult to track important state changes, even though they occur correctly on-chain.

Impact:

- Reduced transparency of state changes
- Difficulty for external systems to index and monitor critical operations
- Potential compliance and auditability issues

Recommendation

Emit appropriate **events** whenever critical arithmetic state updates occur. This aligns with ERC standards and industry best practices.

References

Solidity Events Documentation <https://docs.soliditylang.org/en/latest/contracts.html#events>



SNC-01: Conformity to Solidity naming conventions

Vulnerabilities	Severity	Category	Status
Conformity to Solidity naming conventions	Informational	Coding Style	Acknowledge

Finding Code

```
✗ Parameter PresaleContracts.calculateTokens(uint256)._usdAmount (src/PresaleContracts.sol:401-402) is not in mixedCase
✗ Parameter PresaleContracts.getUserInfo(address)._user (src/PresaleContracts.sol:386-387) is not in mixedCase
✗ Parameter PresaleContracts.lockStage(uint8)._stage (src/PresaleContracts.sol:272) is not in mixedCase
✗ Parameter PresaleContracts.overrideETHPrice(uint256)._price (src/PresaleContracts.sol:287-289) is not in mixedCase
✗ Parameter PresaleContracts.recoverTokens(address,address,uint256)._amount (src/PresaleContracts.sol:349) is not in mixedCase
✗ Parameter PresaleContracts.recoverTokens(address,address,uint256)._to (src/PresaleContracts.sol:349) is not in mixedCase
✗ Parameter PresaleContracts.recoverTokens(address,address,uint256)._token (src/PresaleContracts.sol:349) is not in mixedCase
✗ Parameter PresaleContracts.setClaimingConfig(bool,uint256,uint256)._bonusUnlock (src/PresaleContracts.sol:242) is not in mixedCase
✗ Parameter PresaleContracts.setClaimingConfig(bool,uint256,uint256)._claimStart (src/PresaleContracts.sol:241-242) is not in mixedCase
✗ Parameter PresaleContracts.setClaimingConfig(bool,uint256,uint256)._enabled (src/PresaleContracts.sol:241) is not in mixedCase
✗ Parameter PresaleContracts.setPaused(bool)._paused (src/PresaleContracts.sol:367) is not in mixedCase
✗ Parameter PresaleContracts.setPaymentWallet(address)._wallet (src/PresaleContracts.sol:255) is not in mixedCase
✗ Parameter PresaleContracts.setReferralGasLimit(uint256)._gasLimit (src/PresaleContracts.sol:263-264) is not in mixedCase
✗ Parameter PresaleContracts.setToken(address)._mmvToken (src/PresaleContracts.sol:144-147) is not in mixedCase
✗ Parameter PresaleContracts.updateStage(uint8,uint256,uint256,uint256)._bonusPercent (src/PresaleContracts.sol:304-305) is not in mixedCase
✗ Parameter PresaleContracts.updateStage(uint8,uint256,uint256,uint256)._pricePerTokenUSD (src/PresaleContracts.sol:302-304) is not in mixedCase
✗ Parameter PresaleContracts.updateStage(uint8,uint256,uint256,uint256)._stageIndex (src/PresaleContracts.sol:302) is not in mixedCase
✗ Parameter PresaleContracts.updateStage(uint8,uint256,uint256,uint256)._tokensToSell (src/PresaleContracts.sol:302) is not in mixedCase
✗ Parameter PresaleContracts.withdrawUnsoldTokens(address)._to (src/PresaleContracts.sol:320-321) is not in mixedCase
```

Description

The smart contract does not fully adhere to Solidity's standard naming conventions. Consistent and conventional naming improves readability, maintainability, and helps avoid confusion or misinterpretation of a contract's intent.

Recommendation

It is recommended to refactor the contract to fully comply with Solidity's official naming conventions. This practice aligns with industry standards and facilitates more effective collaboration, auditing, and code comprehension.



References

Solidity Style Guide – Solidity Docs <https://docs.soliditylang.org/en/latest/style-guide.html>



CNN-01: Conformance to Numeric Notation Best Practices (too-many-digits)

Vulnerabilities	Severity	Category	Status
Conformance to Numeric Notation Best Practices (too-many-digits)	Informational	Coding Style	Acknowledge

Finding Code

✗

PresaleContracts.setReferralGasLimit(uint256) (src/PresaleContracts.sol:262-270) uses literals with too many digits:

- require(bool,string)(_gasLimit >= 500000 && _gasLimit <= 5000000,Invalid gas limit) (src/PresaleContracts.sol#264-265)

✗

PresaleContracts.slitherConstructorVariables() (src/PresaleContracts.sol:28-530) uses literals with too many digits:

- referralGasLimit = 400000 (src/PresaleContracts.sol#58-59)

Description

The contract uses **numeric literals with excessive digits** that reduce readability and increase the likelihood of human error. Solidity allows developers to use **underscores (_) as visual separators** in numeric literals to improve clarity, especially for large numbers.

For example, a literal such as 10000000000000000000 is harder to read and verify compared to the more standardized 1_000_000_000_000_000_000.

Non-standard or unclear numeric notation may lead to misinterpretation by developers, reviewers, or auditors, potentially introducing mistakes when updating or verifying values.

Recommendation

Adopt Solidity's numeric literal separators (_) for large numbers to **enhance readability** and ensure conformance with style best practices.

Use:

- *Ether suffix,*
- *Time suffix,* or
- *The scientific notation*

References

Solidity Documentation – Literals <https://docs.soliditylang.org/en/latest/types.html#integer-literals>



UCE-01: Use Custom Errors

Vulnerabilities	Severity	Category	Status
Use Custom Errors	Informational	Gas Optimization	<button>Acknowledge</button>

Finding Code

```
File: PresaleContracts.sol

130: require(IERC20Metadata(_usdtToken).decimals() == 6, "Not USDT");

255: require(_bonusUnlock > _claimStart, "Invalid times");

273: require(_gasLimit >= 50000 && _gasLimit <= 500000, "Invalid gas limit");

302: require(block.timestamp >= lastPriceUpdateTime + 1 hours, "Too soon");
```

Description

The contract uses `require()` statements with revert strings to handle errors and enforce conditions. While this is syntactically valid, it is not the most efficient method in terms of gas consumption.

Starting with **Solidity 0.8.4**, developers can define and use custom errors as a more gas-efficient and structured alternative to revert strings. Custom errors reduce deployment and runtime costs by avoiding the storage of long string literals in contract bytecode.

Recommendation

Refactor `require` statements with revert strings into custom errors, particularly in frequently used functions or large codebases. This approach improves gas efficiency and allows for more structured error handling, enhancing the overall quality and professionalism of the code.

References

Control Structures – Solidity Docs <https://docs.soliditylang.org/en/latest/control-structures.html#custom-errors>



DIV-01: Don't initialize variables with default value

Vulnerabilities	Severity	Category	Status
Don't initialize variables with default value	Informational	Gas Optimization	Acknowledge

Finding Code

```
File: MetaMemeVault.sol  
  
77: for (uint i = 0; i < 5; i++) {  
  
188: for (uint256 i = 0; i < 6; ) {
```

```
File: PresaleContracts.sol  
  
335: bool hasTokensLeft = false;  
  
336: for (uint8 i = 0; i <= 4; i++) {
```

Description

In Solidity, variables are automatically assigned default values when declared. For example, uint and int default to 0, bool defaults to false, and address defaults to 0x000...000. Explicitly assigning these default values during variable declaration is **redundant** and results in **unnecessary bytecode and gas usage** during contract deployment.

Impact:

- Slight increase in contract size and deployment cost
- Reduced code clarity due to redundant assignments
- Missed opportunity for gas optimization

Recommendation

Avoid explicitly initializing variables to their default values. Rely on Solidity's automatic default initialization to reduce bytecode size and deployment gas costs.

Example *This is not fixed code:*

```
uint256 public count; // Automatically initialized to 0
```

References

<https://docs.soliditylang.org/en/latest/style-guide.html#initial-values>



PDM-01: Use of Post-Increment (i++) Instead of Pre-Increment (++i) in Loops

Vulnerabilities	Severity	Category	Status
Use of Post-Increment (i++) Instead of Pre-Increment (++i) in Loops	Informational	Gas Optimization	Acknowledge

Finding Code

```
File: MetaMemeVault.sol

77: for (uint i = 0; i < 5; i++) {
78: for (uint j = i + 1; j < 6; j++) {
```

```
File: PresaleContracts.sol

291: currentStage++;

336: for (uint8 i = 0; i <= 4; i++) {

543: currentStage++;
```

Description

The contract uses the post-increment (i++) or post-decrement (i--) operator in a for-loop. While this is syntactically valid, using pre-increment (++i) or pre-decrement (--i) is marginally more gas-efficient in Solidity.

This is because post-increment creates a temporary copy of the variable before incrementing, whereas pre-increment directly increments the variable, avoiding unnecessary stack operations. Although the gas difference is small, this optimization is considered a best practice, especially within loops that may execute multiple iterations.

Recommendation

Replace all instances of `i++` / `i--` in loop expressions with `++i` / `--i` for slight gas savings and improved efficiency, especially when the return value of the increment/decrement expression is not used.



IRU-01: Inefficient require() Usage with && Conditions

Vulnerabilities	Severity	Category	Status
Inefficient require() Usage with && Conditions	Informational	Gas Optimization	<button>Acknowledge</button>

Finding Code

```
File: PresaleContracts.sol  
  
273: require(_gasLimit >= 50000 && _gasLimit <= 500000, "Invalid gas limit");
```

Description

The contract uses `require()` statements that contain **multiple conditions joined by the logical AND operator (&&)**. While functionally correct, this pattern is slightly less gas-efficient compared to splitting the conditions into **separate `require()` statements**.

This is because Solidity will evaluate **all conditions** in the combined `require()` before generating a revert string, while multiple `require()` statements can short-circuit earlier and provide **clearer error reporting**.

Recommendation

Refactor `require()` statements that combine conditions with `&&` into **separate `require()` calls**. This improves **gas efficiency, clarity, and debugging experience**.



UIC-01: Use != 0 instead of > 0 for unsigned integer comparison

Vulnerabilities	Severity	Category	Status
Use != 0 instead of > 0 for unsigned integer comparison	Informational	Gas Optimization	Acknowledge

Finding Code

```
File: PresaleContracts.sol  
  
123: _maxTokensPerBuyer > 0 &&  
  
337: if (stages[i].tokensToSell > 0) {
```

Description

The contract uses `> 0` to check whether an unsigned integer (uint) is non-zero. While functionally correct, this comparison is slightly less efficient than using `!= 0`, due to the way EVM opcodes are generated and optimized. In the EVM, the `!=` operation is generally compiled to fewer instructions than `>`, resulting in minor gas savings. Since uint variables cannot be negative, both expressions yield the same result, but `!= 0` is considered a more gas-optimal and idiomatic choice for checking non-zero conditions.

Recommendation

Use `!= 0` instead of `> 0` when checking whether a `uint` is non-zero. This is a gas-efficient and stylistically preferred approach in Solidity.



A Appendix

A.1 Source Units in Scope

Source Units Analyzed: 2
Source Units in Scope: 2 (100%)

Type	File	Logic Contracts	Interfaces	Lines	nLines	nSLOC	Comment Lines	Complex. Score	Capabilities
	src/Pre sal eC ont rac ts.s ol	7	6	1411	1165	573	580	460	
	src/Me ta Me me Va ult. sol	5	5	1008	908	352	510	268	
	Tot als	12	11	2419	2073	925	1090	728	



A.2 Visibility, Mutability, Modifier function testing

Components

 ****Contracts	 ****Libraries	 ****Interfaces	 ****Abstract
2	1	11	9

Exposed Functions

This section lists functions that are explicitly declared public or payable. Please note that getter methods for public stateVars are not included.

 ****Public	 ****Payable
80	2

External	Internal	Private	Pure	View
58	112	6	0	47

StateVariables

Total	 ****Public
47	28

Capabilities

Solidity Versions observed	 ** Experimental Features**	 ** Can Receive Funds**	 ** Uses Assembly**	 ** Has Destroyable Contracts**
>=0.4.16 >=0.6.2 ^0.8.20 ^0.8.0 >=0.8.4		yes	yes (2 asm blocks)	***

 ** Transfers ETH**	 **** Low-Level Calls	 ** DelegateCall**	 ** Uses Hash Functions**	 ** ECTRecover**	 ** New/Create/Create2**
yes	***	***	***	***	***

 ** TryCatch**	Σ Unchecked
yes	yes

Dependencies / External Imports

Dependency / Import Path	Count
--------------------------	-------



A.3 Contracts Description Table

Contracts Description Table

Contract****	Type****	Bases****		
└	Function Name	Visibility	Mutability	Modifiers
IERC20	Interface			
└	totalSupply	External !		NO !
└	balanceOf	External !		NO !
└	transfer	External !	●	NO !
└	allowance	External !		NO !
└	approve	External !	●	NO !
└	transferFrom	External !	●	NO !
IERC20Metadata	Interface	IERC20		
└	name	External !		NO !
└	symbol	External !		NO !
└	decimals	External !		NO !
IERC165	Interface			
└	supportsInterface	External !		NO !
IERC1363	Interface	IERC20, IERC165		
└	transferAndCall	External !	●	NO !
└	transferAndCall	External !	●	NO !
└	transferFromAndCall	External !	●	NO !
└	transferFromAndCall	External !	●	NO !
└	approveAndCall	External !	●	NO !
└	approveAndCall	External !	●	NO !
SafeERC20	Library			



Contract****	Type****	Bases****		
L	safeTransfer	Internal 🔒	●	
L	safeTransferFrom	Internal 🔒	●	
L	trySafeTransfer	Internal 🔒	●	
L	trySafeTransferFrom	Internal 🔒	●	
L	safeIncreaseAllowance	Internal 🔒	●	
L	safeDecreaseAllowance	Internal 🔒	●	
L	forceApprove	Internal 🔒	●	
L	transferAndCallRelaxed	Internal 🔒	●	
L	transferFromAndCallRelaxed	Internal 🔒	●	
L	approveAndCallRelaxed	Internal 🔒	●	
L	_callOptionalReturn	Private 🔒	●	
L	_callOptionalReturnBool	Private 🔒	●	
Context	Implementation			
L	_msgSender	Internal 🔒		
L	_msgData	Internal 🔒		
L	_contextSuffixLength	Internal 🔒		
Ownable	Implementation	Context		
L		Public !	●	NO !
L	owner	Public !		NO !
L	_checkOwner	Internal 🔒		
L	renounceOwnership	Public !	●	onlyOwner
L	transferOwnership	Public !	●	onlyOwner
L	_transferOwnership	Internal 🔒	●	
Ownable2Step	Implementation	Ownable		
L	pendingOwner	Public !		NO !



Contract****	Type****	Bases****		
└	transferOwnership	Public !	●	onlyOwner
└	_transferOwnership	Internal 🔒	●	
└	acceptOwnership	Public !	●	NO !
ReentrancyGuard	Implementation			
└		Public !	●	NO !
└	_nonReentrantBefore	Private 🔒	●	
└	_nonReentrantAfter	Private 🔒	●	
└	_reentrancyGuardEntered	Internal 🔒		
Pausable	Implementation	Context		
└		Public !	●	NO !
└	paused	Public !		NO !
└	_requireNotPaused	Internal 🔒		
└	_requirePaused	Internal 🔒		
└	_pause	Internal 🔒	●	whenNotPaused
└	_unpause	Internal 🔒	●	whenPaused
AggregatorV3Interface	Interface			
└	decimals	External !		NO !
└	description	External !		NO !
└	version	External !		NO !
└	getRoundData	External !		NO !
└	latestRoundData	External !		NO !
PresaleContracts	Implementation	Ownable2Step, ReentrancyGuard, Pausable		
└		Public !	●	Ownable
└	setToken	External !	●	onlyOwner



Contract****	Type****	Bases****		
L	buyWithETH	External !	🟢	nonReentrant whenNotPaused
L	buyWithUSDT	External !	🔴	nonReentrant whenNotPaused
L	claimBaseTokens	External !	🔴	nonReentrant
L	claimBonusTokens	External !	🔴	nonReentrant
L	setClaimingConfig	External !	🔴	onlyOwner
L	setPaymentWallet	External !	🔴	onlyOwner
L	setReferralGasLimit	External !	🔴	onlyOwner
L	lockStage	External !	🔴	onlyOwner
L	advanceStage	External !	🔴	onlyOwner
L	overrideETHPrice	External !	🔴	onlyOwner
L	updateStage	External !	🔴	onlyOwner
L	withdrawUnsoldTokens	External !	🔴	onlyOwner
L	recoverTokens	External !	🔴	onlyOwner
L	setPaused	External !	🔴	onlyOwner
L	getCurrentStageInfo	External !		NO !
L	getUserInfo	External !		NO !
L	calculateTokens	External !		NO !
L	getPresaleStats	External !		NO !
L	getUnclaimedTokens	External !		NO !
L	_getETHPrice	Private 🗝️	🔴	
L	_processPurchase	Private 🗝️	🔴	
IReferralRewards	Interface			
L	registerReferral	External !	🔴	NO !
IERC20	Interface			
L	totalSupply	External !		NO !
L	balanceOf	External !		NO !



Contract****	Type****	Bases****		
L	transfer	External !	●	NO !
L	allowance	External !		NO !
L	approve	External !	●	NO !
L	transferFrom	External !	●	NO !
IERC20Metadata	Interface	IERC20		
L	name	External !		NO !
L	symbol	External !		NO !
L	decimals	External !		NO !
Context	Implementation			
L	_msgSender	Internal 🔒		
L	_msgData	Internal 🔒		
L	_contextSuffixLength	Internal 🔒		
IERC20Errors	Interface			
IERC721Errors	Interface			
IERC1155Errors	Interface			
ERC20	Implementation	Context, IERC20, IERC20Metadata, IERC20Errors		
L		Public !	●	NO !
L	name	Public !		NO !
L	symbol	Public !		NO !
L	decimals	Public !		NO !
L	totalSupply	Public !		NO !
L	balanceOf	Public !		NO !
L	transfer	Public !	●	NO !
L	allowance	Public !		NO !



Contract****	Type****	Bases****		
└	approve	Public !	●	NO !
└	transferFrom	Public !	●	NO !
└	_transfer	Internal 🔒	●	
└	_update	Internal 🔒	●	
└	_mint	Internal 🔒	●	
└	_burn	Internal 🔒	●	
└	_approve	Internal 🔒	●	
└	_approve	Internal 🔒	●	
└	_spendAllowance	Internal 🔒	●	
Ownable	Implementation	Context		
└		Public !	●	NO !
└	owner	Public !		NO !
└	_checkOwner	Internal 🔒		
└	renounceOwnership	Public !	●	onlyOwner
└	transferOwnership	Public !	●	onlyOwner
└	_transferOwnership	Internal 🔒	●	
Ownable2Step	Implementation	Ownable		
└	pendingOwner	Public !		NO !
└	transferOwnership	Public !	●	onlyOwner
└	_transferOwnership	Internal 🔒	●	
└	acceptOwnership	Public !	●	NO !
MetaMemeVault	Implementation	ERC20, Ownable2Step		
└		Public !	●	ERC20 Ownable
└	recoverEth	External !	●	onlyOwner
└	recoverToken	External !	●	onlyOwner
└	burn	External !	●	NO !



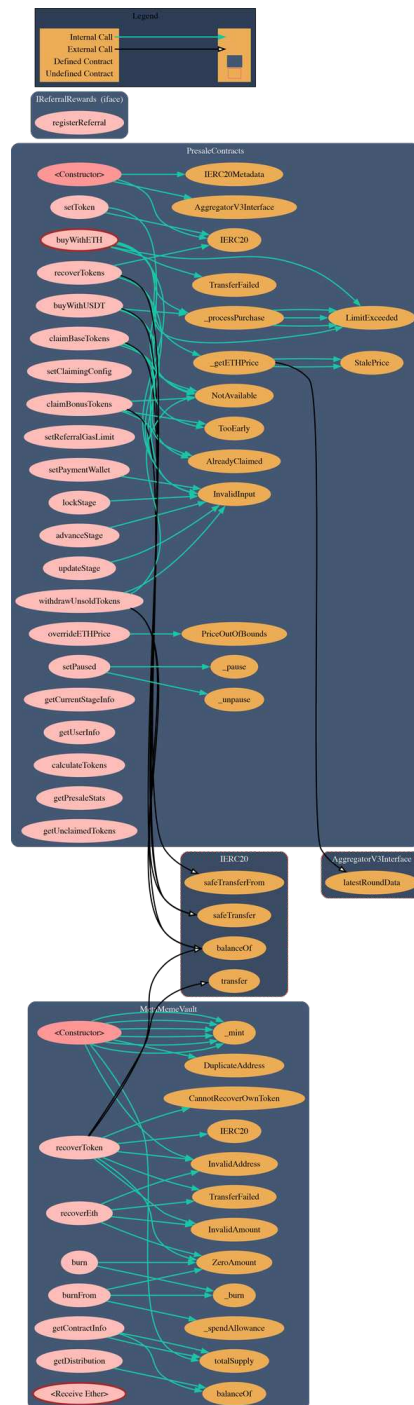
Contract****	Type****	Bases****		
L	burnFrom	External !	●	NO !
L	getDistribution	External !		NO !
L	getContractInfo	External !		NO !
L		External !	👛	NO !

Legend

Symbol****	Meaning****
●	Function can modify state
👛	Function is payable



A.4 Call Graph



A.5 UML Class Diagram



About SCRL

For partner

SCRL

Proven Cybersecurity Expertise

Highlight



Cyber-attacks are ineffective when our customers utilize our services.

Our customers enjoy the highest level of cybersecurity, and we have successfully prevented any significant cyber attacks against them. Over the past three years, we have dedicated significant efforts to safeguarding our customers' data and systems.



Cybersecurity Assessment and Penetration Testing by Multiple Experts

Our team comprises experienced security researchers and penetration testers with expertise in Web3 and Blockchain, as well as general industries. Our primary focus is conducting comprehensive security assessments to provide secure solutions to our valued customers.



Collaborative law enforcement investigations

We have provided assistance to victims in numerous cases, including the Pig-butcher Scam, Hybrid Scam. Through collaborative efforts with law enforcement agencies, we have successfully secured justice through the court system.

SCRL is a dynamic cybersecurity team that provides security assessments, penetration testing, and incident investigations. We utilize specialized tools, expertise, and frameworks to enhance security measures. Additionally, we develop robust internal tools tailored to our organization's needs.

SCRL is committed to driving security for the blockchain industry and businesses alike.

Follow Us On:

Platform	Link
Website	https://scrl.io/
Twitter	https://twitter.com/scrl_io
Telegram	https://t.me/scrl_io
Medium	https://scrl.medium.com/

